



机器视觉实验 2

姓名：冯怡然

学号：2411434

班级：人工智能特色班

日期：2026 年 3 月 30 日

南开大学 · 机器视觉实验

1 题目一：直方图均衡化

1.1 实验目的

1. 深入理解图像直方图的概念及其在数字图像处理中的意义。
2. 掌握直方图均衡化 (Histogram Equalization) 的数学原理及算法流程。
3. 学习并实践如何使用 C++ 与 OpenCV 库进行图像的读取、灰度处理以及像素级操作。
4. 能够编写独立的函数实现直方图均衡化，并分析处理前后的图像对比度变化。

1.2 实验原理

直方图均衡化是一种增强图像对比度的方法。其核心思想是通过一个非线性的映射函数，将原始图像的灰度直方图由集中分布变换为在全亮度级范围内的均匀分布。

对于一张大小为 $M \times N$ ，亮度级为 $k = 256$ 的图像，其具体数学原理如下：

1. **统计原始直方图 H** ：统计图像中每个像素值 p 出现的次数。
2. **计算累计直方图 H_c** ：

$$H_c[p] = \sum_{i=0}^p H[i]$$

其中 $p \in [0, k - 1]$ 。

3. **建立查找表 T** ：通过线性变换将累计分布函数映射到 $[0, k - 1]$ 空间：

$$T[p] = \text{round} \left(\frac{k - 1}{MN} \cdot H_c[p] \right)$$

4. **映射变换**：利用查找表 T 对原图像的每个像素进行灰度重排。

1.3 实验步骤

1. **环境准备**：配置 Visual Studio 开发环境，正确链接 OpenCV 库（确保包含目录、库目录及附加依赖项设置正确）。
2. **图像读取与预处理**：
 - 使用 `imread()` 函数读取 `ratatouille.png` 图像。
 - 使用 `cvtColor()` 将彩色图转换为灰度图。
3. **实现核心算法**：编写 `myEqualizeHist` 函数，严格按照算法流程：
 - 初始化长度为 256 的数组 H 为 0。
 - 遍历图像像素填充 H 。
 - 递归或循环计算累计直方图 H_c 。
 - 根据公式计算查找表 T 。
4. **结果显示**：
 - 引入 `Windows.h` 中的 `SetProcessDPIAware()` 解决高分屏缩放问题。
 - 使用 `namedWindow(..., WINDOW_NORMAL)` 控制显示比例，确保图像完整且不变形。
 - 同时输出原图、灰度图与均衡化后的结果图进行对比。

1.4 程序代码 (C++)

1.4.1 main.cpp

```
1 #include <iostream>
2 #include <fstream>
3 #include "opencv2/opencv.hpp"
4 #include "opencv2/imgproc/imgproc.hpp"
5 #include "opencv2/highgui/highgui.hpp"
6 #include <stdio.h>
7 #include <Windows.h>
8 using namespace cv;
9 using namespace std;
10
11 Mat myEqualizeHist(Mat img)
12 {
13     Mat EqualizedImg;
14     //形成图像直方图
15     int hist[256] = { 0 }; //灰度级为256
16     int pixelNum = img.rows * img.cols; //图像像素总数
17     //统计每个灰度级的像素数
18     for (int r = 0; r < img.rows; r++) {
19         for (int c = 0; c < img.cols; c++) {
20             uchar intensity = img.at<uchar>(r, c);
21             hist[intensity]++;
22         }
23     }
24     //计算累计分布函数
25     int Hc[256] = { 0 };
26     Hc[0] = hist[0];
27     for (int i = 1; i < 256; i++) {
28         Hc[i] = Hc[i - 1] + hist[i];
29     }
30     //计算变换函数
31     uchar T[256] = { 0 };
32     for (int i = 0; i < 256; i++) {
33         T[i] = (uchar)cvRound(((double)Hc[i] * 255.0 / pixelNum));
34     }
35     //重新扫描图像，根据查找表获得变换结果
36     EqualizedImg = img.clone();
37     for (int r = 0; r < img.rows; r++) {
38         for (int c = 0; c < img.cols; c++) {
39             uchar intensity = img.at<uchar>(r, c);
40             EqualizedImg.at<uchar>(r, c) = T[intensity];
41         }
42     }
43     //返回原图像经过直方图均衡化后的变换结果
44     return EqualizedImg;
45 }
46 void main()
```

```

47 {
48     SetProcessDPIAware();
49     Mat input = imread(R"(C:\Users\AliceJFeng\Desktop\ratatouille.png)");
50
51     Mat gray;
52     //彩色图转为灰度图
53     cvtColor(input, gray, COLOR_BGR2GRAY);
54
55     //直方图均衡化，需编程实现
56     Mat EqualizedImg = myEqualizeHist(gray);
57
58     int displayWidth = 600;
59     double scale = (double)displayWidth / input.cols;
60     int displayHeight = (int)(input.rows * scale);
61
62     double zoom = 0.7;
63     Mat showInput, showGray, showEqual;
64
65     resize(input, showInput, Size(), zoom, zoom);
66     resize(gray, showGray, Size(), zoom, zoom);
67     resize(EqualizedImg, showEqual, Size(), zoom, zoom);
68
69     // 4. 直接显示（因为已经 resize 过了，默认窗口就能看全）
70     imshow("1.Original", showInput);
71     imshow("2.Gray", showGray);
72     imshow("3.Equalized", showEqual);
73     Ptr<CLAHE> clahe = createCLAHE();
74     clahe->setClipLimit(2.0);
75     clahe->setTilesGridSize(Size(8, 8));
76     Mat claheImg;
77     clahe->apply(gray, claheImg);
78     imshow("CLAHE Result (More Natural)", claheImg);
79     waitKey(0);
80 }

```

Listing 1: main.cpp

1.5 实验结果显示

原图片如下图所示



图 1: 原图片



图 2: Gray



图 3: EqualizedImg



图 4: CLAHE Result

1.6 实验分析总结

通过实验观察，直方图均衡化能够显著增强图像的整体对比度，特别是对于原本背景较暗、细节不明显的图像，均衡化后图像暗部的层次感得到了提升。

然而，在实验过程中也发现了一些局限性：

- **过曝现象**：对于原本亮度分布较广或存在大面积亮色区域的图像（如 `ratatouille.png` 的高光部分），均衡化可能导致像素值过度向 255 集中，造成局部细节丢失和“过曝”感。
- **噪声增强**：算法在拉伸动态范围的同时，也放大了背景中的随机噪声。

总结而言，全局直方图均衡化适用于图像对比度极低且背景单一的情况。而对于更复杂的图像，考虑引入 CLAHE（限制对比度自适应直方图均衡化）以获得更自然的效果。

2 题目三——中值滤波 (Matlab)

2.1 实验目的

- 掌握中值滤波 (Median Filtering) 的基本原理及其在图像处理中的应用。
- 能够使用 Matlab 编写自定义函数实现中值滤波算法，不依赖系统内置函数。
- 学习如何在图像中添加椒盐噪声 (Salt and Pepper Noise)，并验证中值滤波对该噪声的去除效果。

2.2 实验原理

2.2.1 中值滤波算法

中值滤波是一种非线性平滑技术。其核心思想是将数字图像中某点的值设置为该点邻域窗口内所有像素灰度值的中值。对于一个大小为 $n \times n$ 的掩模 (Mask)，其处理过程可表示为：

$$g(x, y) = \text{median}\{f(x - i, y - j) \mid (i, j) \in W\}$$

其中， $f(x, y)$ 为原始像素值， $g(x, y)$ 为处理后的像素值， W 为定义的邻域范围。

2.2.2 对椒盐噪声的抑制

椒盐噪声由图像中随机出现的纯白点（盐）或纯黑点（椒）组成。由于这些噪声值通常是邻域内的极值，在经过排序取中值的操作后，这些极端值会被剔除，取而代之的是邻域内代表性较强的正常灰度值。

2.3 实验步骤

1. **图像预处理**：读取原始图像，并将其转换为灰度图像，获取图像尺寸。
2. **添加噪声**：使用 `imnoise` 函数向图像中注入密度为 0.05 的椒盐噪声。
3. **编写中值滤波函数**：
 - 确定 3×3 掩模。
 - 使用双重循环遍历图像，避开最外圈边缘像素。
 - 对掩模内的 9 个像素值进行升序排序。
 - 取排序序列的第 5 个值（中值）赋给输出图像对应位置。
4. **结果对比**：显示原图、噪声图及滤波后的结果图。

2.4 程序代码 (Matlab)

2.4.1 MyMedfilt2.m

```
1 function Medfilt2_result=MyMedfilt2(img)
2 img=double(img);
3 [rows,cols]=size(img);
4 res=img;
5 for i=2:rows-1
6     for j=2:cols-1
7         temp_windows = img(i-1:i+1, j-1:j+1);
8         sorted_values=sort(temp_windows(:));
9         res(i,j)=sorted_values(5);
10    end
11 end
12 Medfilt2_result=uint8(res);
13 end
```

Listing 2: MyMedfilt2.m

2.4.2 main.m

```
1 clc;clear all;close all;
2 img=imread('ratatouille.png');
3 % 彩色图转为灰度图
4 img=rgb2gray(img);
5 noisy_img = imnoise(img, 'salt & pepper', 0.05);
6 %中值滤波，需编程实现
7 Medfilt2_result = MyMedfilt2(noisy_img);
8 figure; % 开启新窗口
9 imshow(img);
10 title('Original Gray Image');
11 figure;
12 imshow(noisy_img);
13 title('Salt & Pepper Noise');
14 figure;
15 imshow(Medfilt2_result);
16 title('Median Filter Result');
```

Listing 3: main.m

2.5 实验结果显示

原图片如下图所示



图 5: 原灰度图

原图加入椒盐噪声处理:



图 6: 原灰度图 + 椒盐噪声

运行结果如图所示:



图 7: 噪声中值滤波处理结果

2.6 实验分析总结

- **去噪效果：**实验结果显示，中值滤波能极大地消除椒盐噪声。与均值滤波相比，中值滤波在去除孤立噪声点的同时，较好地保留了图像的边缘轮廓，没有产生明显的运动模糊感。
- **类型处理的重要性：**在实验初期出现的“图像全白”现象证明了类型转换的重要性。通过将计算结果强制转换为 `uint8` 类型，解决了灰度范围溢出的显示问题。